

# RE101 Lab Walkthrough

I searched the malware000 file, and when scrolling saw this encrypted text, which obviously could be something, it is a base64 string because of the dash at the back.

```
[J\A\A]A^A_  
ZmxhZzwwb3BzX2lfdXNlZF8xMzM3X2I2NF9lbnNyeXB0aW9uPgo=  
If you see this message ...  
/dev/urandom
```

```
flag<0ops_i_used_1337_b64_encryption>
```

After decrypting it at cyberchef, I got the original human readable string.

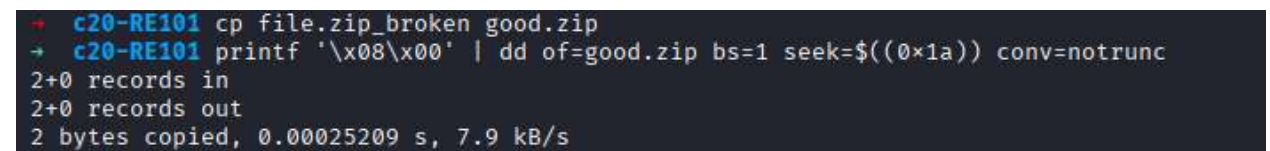
When strings the just\_some\_js file there some strange language, the strange language is a jsfuck

[illegible]

Going to <https://www.dcode.fr/> website to decode the jsfuck to become readable.

[illegible]

```
+ c20-RE101 strings this_is_not_js  
+++++++[>+>+++>++++++>+++++++<<<-]>>>+.+++++.-----  
.←-----.)_-.+.---.<+++++++>+++←-----.)_--..---.++
```



58 58

After repairing the zip, now get the zip hash and send it to john to identify the password

```

→ c20-RE101 unzip -P password good.zip
Archive:  good.zip
  extracting: flag.txt
→ c20-RE101 ls
backup.zip  file.zip_broken  fixed.zip  fla
→ c20-RE101 cat flag.txt
flag<R3ad_th3_spec>

```

Unzip the fixed file with found password and read the file

## MALWARE101

```

c20-RE101 strings -tx malware101
238 /lib64/ld-linux-x86-64.so.2
2f9 libc.so.6
303 printf
30a __libc_start_main
31c GLIBC_2.2.5
328 __gmon_start__
5c0 AWAVI
5c7 AUATL
61a [A\A]A^A_
644 Nothing to see here....
717 ;*3$"
1030 GCC: (Ubuntu 7.5.0-3ubuntu1~18.04) 7.5.0
1059 clang version 6.0.0-1ubuntu2 (tags/RELEASE_600/final)
1090 .shstrtab

```

Looks like strings doesn't show the file behaviour, so we use ghidra to go deeper

```
0, stripped
```

When checking file malware101, it shows stripped which means most debugging symbol information is removed. Instead of seeing useful names, Ghidra may initially generate names such as

|   |   |              |
|---|---|--------------|
| > | f | entry        |
| > | f | FUN_004003c0 |
| > | f | FUN_00400410 |
| > | f | FUN_00400420 |
| > | f | FUN_00400450 |
| > | f | FUN_004005c0 |
| > | f | FUN_00400630 |

Usually when reverse engineering using ghidra, we first look at the entry function first to see how the file code started executing.

```
1
2 void processEntry entry(undefined8 param_1,undefined8 param_2)
3
4 {
5     undefined1 auStack_8 [8];
6
7     __libc_start_main(main,param_2,&stack0x00000008,FUN_004005c0,FUN_00400630,param_1,auStack_8);
8     do {
9         /* WARNING: Do nothing block with infinite loop */
10    } while( true );
11 }
```

Seeing the decompile, we can see after *libc\_start\_main*, calls a function which i rename it to main

```
1 undefined8 main(void)
2
3 {
4     printf("Nothing to see here....\n");
5     return 0;
6 }
```

Looking here, the decompile show the string output we just saw but still nothing interesting yet, now we go to the listing view which show something interesting

```

*****
*                                     *
*                                     *
*****
FUNCTION
*****
undefined main()
undefined  ⚠️<UNASSIGNED> <RETURN>
undefined4  Stack[-0xc]:4 local_c          XREF[1]: 004004e5 (W)
undefined1  Stack[-0x1a0... local_la0     XREF[1]: 0040051d (W)
undefined1  Stack[-0x1a1... local_la1     XREF[1]: 00400516 (W)
undefined1  Stack[-0x1a2... local_la2     XREF[1]: 0040057f (W)
undefined1  Stack[-0x1a3... local_la3     XREF[1]: 00400578 (W)
undefined1  Stack[-0x1a4... local_la4     XREF[1]: 00400594 (W)
undefined1  Stack[-0x1a5... local_la5     XREF[1]: 0040056a (W)
undefined1  Stack[-0x1a6... local_la6     XREF[1]: 00400547 (W)
undefined1  Stack[-0x1a7... local_la7     XREF[1]: 0040052b (W)
undefined1  Stack[-0x1a8... local_la8     XREF[1]: 004004ec (W)
undefined1  Stack[-0x1a9... local_la9     XREF[1]: 00400508 (W)
undefined1  Stack[-0x1aa... local_laa     XREF[1]: 00400501 (W)
undefined1  Stack[-0x1ab... local_lab     XREF[1]: 004004fa (W)
undefined1  Stack[-0x1ac... local_lac     XREF[1]: 00400563 (W)
undefined1  Stack[-0x1ad... local_lad     XREF[1]: 0040055c (W)
undefined1  Stack[-0x1ae... local_lae     XREF[1]: 00400540 (W)
undefined1  Stack[-0x1af... local_laf     XREF[1]: 00400524 (W)
undefined1  Stack[-0x1b0... local_lb0     XREF[1]: 00400571 (W)
undefined1  Stack[-0x1b1... local_lb1     XREF[1]: 004004f3 (W)
undefined1  Stack[-0x1b2... local_lb2     XREF[1]: 00400539 (W)
undefined1  Stack[-0x1b3... local_lb3     XREF[1]: 0040050f (W)
undefined1  Stack[-0x1b4... local_lb4     XREF[1]: 0040058d (W)
undefined1  Stack[-0x1b5... local_lb5     XREF[1]: 00400586 (W)
undefined1  Stack[-0x1b6... local_lb6     XREF[1]: 00400532 (W)
undefined1  Stack[-0x1b7... local_lb7     XREF[1]: 0040054e (W)
undefined1  Stack[-0x1b8... local_lb8     XREF[1]: 00400555 (W)
undefined4  Stack[-0x1bc... local_lbc     XREF[1]: 004005a4 (W)
main
XREF[3]: entry:004003fd(*), 00400684,
00400728(*)
004004d0 55          PUSH          RBP

```

Which show this function which we need to read from bottom to top, that is how we arrange the the each character to become a full string.



```

dword ptr [RBP + local_c],0x0
byte ptr [RBP + local_1a8],0x67
byte ptr [RBP + local_1b1],0x61
byte ptr [RBP + local_1ab],0x72
byte ptr [RBP + local_1aa],0x69
byte ptr [RBP + local_1a9],0x6e
byte ptr [RBP + local_1b3],0x73
byte ptr [RBP + local_1a1],0x3e
byte ptr [RBP + local_1a0],0x0
byte ptr [RBP + local_1af],0x6b
byte ptr [RBP + local_1a7],0x73
byte ptr [RBP + local_1b6],0x61
byte ptr [RBP + local_1b2],0x54
byte ptr [RBP + local_1ae],0x5f
byte ptr [RBP + local_1a6],0x5f
byte ptr [RBP + local_1b7],0x6c
byte ptr [RBP + local_1b8],0x66
byte ptr [RBP + local_1ad],0x73
byte ptr [RBP + local_1ac],0x74
byte ptr [RBP + local_1a5],0x4c
byte ptr [RBP + local_1b0],0x43
byte ptr [RBP + local_1a3],0x41

```

We double click then it bring us to see the reference of each local\_, which the green text convert hex into character, this is called stack strings, after convert the full string will look like this,

flag<sTaCk\_strings\_LMAO>\0

## MALWARE201

This is the main function, i already rename some of them

```
1 undefined8 main(void)
2
3
4 {
5     size_t sVar1;
6     void *__ptr;
7
8     printf("The encrypted flag is: \n");
9     print_something(&encrypted_flag,0x20);
10    printf("\n\n\n");
11    sVar1 = strlen("my message");
12    printf("encrypt(\"%s\", %ld) == \n", "my message", sVar1);
13    __ptr = (void *)encrypt("my message", sVar1);
14    print_something(__ptr, sVar1);
15    printf("\n\n");
16    free(__ptr);
17    return 0;
18 }
```

```
1 2 example: print_hex_bytes (malware201)
2 void print_hex_bytes(long data,ulong length)
3
4 {
5     ulong i;
6
7     for (i = 0; i < length; i = i + 1) {
8         printf("\x%02x", (ulong)*(byte *) (data + i));
9     }
10    return;
11 }
```

This the print function

|                                     |          |    |                            |   |          |  |                                                            |
|-------------------------------------|----------|----|----------------------------|---|----------|--|------------------------------------------------------------|
| 50 52 70 00                         |          |    |                            |   | XREF[3]: |  | main:004006d2(*),<br>main:004006dc(*),<br>main:004006ee(*) |
| encrypted_flag                      |          |    |                            |   |          |  |                                                            |
| 0040082b                            | 6d       | ?? | 6Dh                        | m |          |  |                                                            |
| 0040082c                            | 78       | ?? | 78h                        | x |          |  |                                                            |
| 0040082d                            | 61       | ?? | 61h                        | a |          |  |                                                            |
| 0040082e                            | 6c       | ?? | 6Ch                        | l |          |  |                                                            |
| 0040082f                            | dd       | ?? | DDh                        |   |          |  |                                                            |
| 00400830                            | 7e       | ?? | 7Eh                        | ~ |          |  |                                                            |
| 00400831                            | 65       | ?? | 65h                        | e |          |  |                                                            |
| 00400832                            | 7e       | ?? | 7Eh                        | ~ |          |  |                                                            |
| 00400833                            | 47       | ?? | 47h                        | G |          |  |                                                            |
| 00400834                            | 6a       | ?? | 6Ah                        | j |          |  |                                                            |
| 00400835                            | 4f       | ?? | 4Fh                        | O |          |  |                                                            |
| 00400836                            | cc       | ?? | CCh                        |   |          |  |                                                            |
| 00400837                            | f7       | ?? | F7h                        |   |          |  |                                                            |
| 00400838                            | ca       | ?? | CAh                        |   |          |  |                                                            |
| 00400839                            | 73       | ?? | 73h                        | s |          |  |                                                            |
| 0040083a                            | 68       | ?? | 68h                        | h |          |  |                                                            |
| 0040083b                            | 55       | ?? | 55h                        | U |          |  |                                                            |
| 0040083c                            | 42       | ?? | 42h                        | B |          |  |                                                            |
| 0040083d                            | 53       | ?? | 53h                        | S |          |  |                                                            |
| 0040083e                            | dc       | ?? | DCh                        |   |          |  |                                                            |
| 0040083f                            | d7       | ?? | D7h                        |   |          |  |                                                            |
| 00400840                            | d4       | ?? | D4h                        |   |          |  |                                                            |
| 00400841                            | 6b       | ?? | 6Bh                        | k |          |  |                                                            |
| 00400842                            | ec       | ?? | ECh                        |   |          |  |                                                            |
| 00400843                            | db       | ?? | DBh                        |   |          |  |                                                            |
| 00400844                            | d2       | ?? | D2h                        |   |          |  |                                                            |
| 00400845                            | e1       | ?? | E1h                        |   |          |  |                                                            |
| 00400846                            | 1c       | ?? | 1Ch                        |   |          |  |                                                            |
| 00400847                            | 6d       | ?? | 6Dh                        | m |          |  |                                                            |
| 00400848                            | de       | ?? | DEh                        |   |          |  |                                                            |
| 00400849                            | d1       | ?? | D1h                        |   |          |  |                                                            |
| 0040084a                            | c2       | ?? | C2h                        |   |          |  |                                                            |
| 0040084b                            | 00       | ?? | 00h                        |   |          |  |                                                            |
| s_The_encrypted_flag_is: "_0040084c |          |    |                            |   | XREF[1]: |  | main:004006c8(*)                                           |
| 0040084c                            | 54 68 65 | ds | "The encrypted flag is: \" |   |          |  |                                                            |

So we turn those hex numbers into ASCII



```
Decompile: encrypt - (malware201)
1
2 void * encrypt(long input, size_t length)
3
4 {
5     void *output;
6     undefined8 i;
7     undefined1 auVar1 [16];
8
9     output = calloc(length, 1);
10    for (i = 0; i < length; i = i + 1) {
11        auVar1._8_8_ = 0;
12        auVar1._0_8_ = i;
13        *(byte *)((long)output + i) =
14            (*(char *) (input + i) << 1 | 1U) ^ (SUB161(auVar1 % ZEXT816(0xff), 0) | 0xa0);
15    }
16    return output;
17 }
18
```

This is the encryption function which show how it is encrypted,

```
encrypted = bytes.fromhex("""
6d 78 61 6c dd 7e 65 7e
47 6a 4f cc f7 ca 73 68
55 42 53 dc d7 d4 6b ec
db d2 e1 1c 6d de d1 c2
""")

decrypted = bytearray()

for i, c in enumerate(encrypted):
    key = (i % 0xff) | 0xa0

    # Undo XOR
    temp = c ^ key

    # Undo left shift
    plain = temp >> 1

    decrypted.append(plain)

print(decrypted.decode())
```

So the decrypt script is above to reverse the encrypt function.

flag<malwar3-3ncryp710n-15-Sh17>